	Scenario: ~206.8M of 64 bytes (ethereum accounts)		**Scenario:** 2^32 of 64 bytes (entire ethereum state)	
	Latency	Throughput	Latency	Throughput
Status quo				
Ethereum's InsPIRe	~250ms	40-60 queries/sec/gpu	5.2 secs (linear scanning can possibly parallelized away, no longer fits in a single GPU)	2-3 queries/sec (projected, cannot be parallelized away, real number can be worse due to not fitting in a single GPU)
NeoMap: Oblivious Labs' TEE-based ORAM	30 us (non-batched) 10-20ms (batched)	33K queries/sec/cpu 300K-400K queries/sec/cpu	50 us (non-batched) 50ms (batched)	20K 33K
Garbled RAM (our preliminary work)	~400ms (extrapolated)	1/latency (currently no throughput optimization)	~700ms (extrapolated)	1/latency (currently no throughput optimization)
Proposed work				
MicrORAM: faster TEE-based ORAM (desired target of proposed work)	<2 us (non-batched) < 500 us (batched)	>2000K queries/sec/cpu (batched)	<3us	>1000K queries/sec/cpu (batched)
Garbled RAM (desired target of our proposed work)	~100ms	40-60 queries/sec/cpu	~150ms	50-70 queries/sec/cpu

Oblivious RAM (ORAM) constructions primarily fall into two categories: hierarchical and tree-based. Hierarchical ORAM organizes records into hash tables of exponentially increasing size. Every access writes the requested record to the smallest table, requiring periodic rebuilds into larger tables as smaller ones fill up. In contrast, tree-based ORAM avoids periodic rebuilds altogether; each access performs a fixed amount of maintenance (termed eviction) and retrieves the record's location using a position map. Tree-based ORAM has traditionally been favored for performance because hierarchical ORAM's rebuild process introduces two major drawbacks: high computational overhead (with state-of-the-art approaches like H2O2RAM relying on expensive oblivious sorting at $O(\log^3 N)$ work per access) and high tail latency, as incoming requests must pause during large rebuilds. We observe, however, that these limitations stem from specific implementation choices rather than the hierarchical paradigm itself. While an attacker can observe memory access patterns, data movement within internal processor registers remains hidden. We hence utilize AVX-512 vector instructions to design a rebuild algorithm that reduces computational complexity by a logarithmic factor compared to oblivious sorting. Together with other optimizations, we achieve a 80~240 times concrete speedup in rebuild. Furthermore, to solve the tail latency problem, we implement new deamortization techniques to execute rebuild operations concurrently in the background, ensuring uninterrupted request processing.

AVX-512 register-level optimizations can similarly accelerate tree-based ORAM when managing large entries, such as nodes in Ethereum's Merkle Patricia Trie. In our hybrid architecture, the hierarchical ORAM serves as the position map for the tree-based ORAM. A unified interface dynamically selects the hierarchical scheme for small entries and the tree-based scheme for large ones.

We use a hybrid approach, entry size is large, position map cost kind of gets absorbed.
In tree based oram, we want bucket (tree node) to be large enough to span multiple cachelines
Avx512 makes movement of payloads cheap. When entry size is smaller, the computation of the eviction is dominating.

==========

Faster Garbled RAM:

\paragraph{Extending garbled RAM to multiple parties.}

Our PicoGRAM/Zebra has a 2-party garbled RAM. For Ethereum deployment scenario, we will initially aim for a scenario with 10-20 parties. We will work on extending it to multiple parties. Our CRYPTO'26 paper~\cite{} already developed the cryptographic groundwork, but the CRYPTO'26 paper only instantiated it for circuits. The cryptographic techniques developed in the CRYPTO'26 work makes multi-party arithmetic garbling practically efficient which previously lived in more theory form.
Therefore, the next step is to combine the cryptographic techniques in our CRYPTO'26 paper with the RAM techniques in our PicoGRAM/Zebra paper. The new efficient arithmetic garbling will allow us to improve the bandwidth for moderately sized blocks, e.g. in Ethereum's setting

\paragraph{Faster preprocessing.}

In our current Garbled RAM, there is a preprocessing phase that accomplishes

- Multiplication triples
- garbling

The processing can be done offline and it's infinitely parallelizable, so it does not count towards the online costs. However, it's still worth making the preprocessing more practical since right now, without much effort at optimization, the preprocessing requires at least an order of magnitude more work than the online phase.

Faster preprocessing with Ring-LPN. Our current construction uses LPN-based pseudorandom correlation generators (PCGs) to produce authenticated randomness and multiplication triples for preprocessing. While this minimizes communication overhead, standard LPN local expansion is computationally heavy, making preprocessing a bottleneck that requires an order of magnitude more work than the online phase. We will replace this component with a Ring-LPN-based PCG, which leverages polynomial-ring structures to generate correlated batches far more efficiently. Prior work demonstrates an order-of-magnitude speedup—generating ~50,000 triples/sec per core compared to ~7,000—while reducing seed material from gigabytes to megabytes (https://eprint.iacr.org/2022/1035.pdf). We will adapt and benchmark Ring-LPN in our framework to remove preprocessing as an end-to-end bottleneck.

\paragraph{GPU and avx512 acceleration.} garbling and preprocessing should benefit from parallelism.